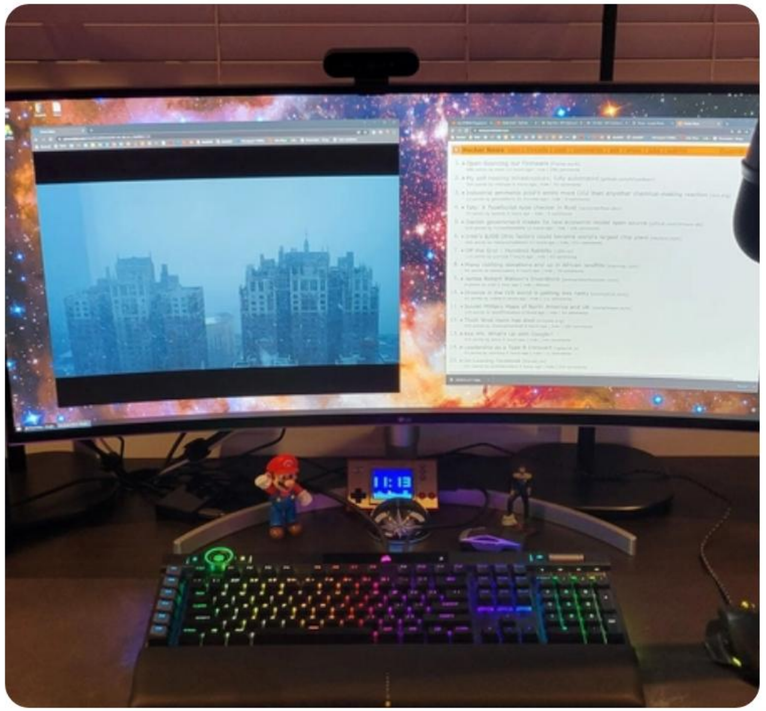
MENTOR CALL

JavaScript Advanced



Mentor: Joseph Kwanusu

Topic: Control Flow & Functions

Non-Primitive Data Types

Unlike primitives, non-primitives can store collections of data and are **passed by reference**.

Objects

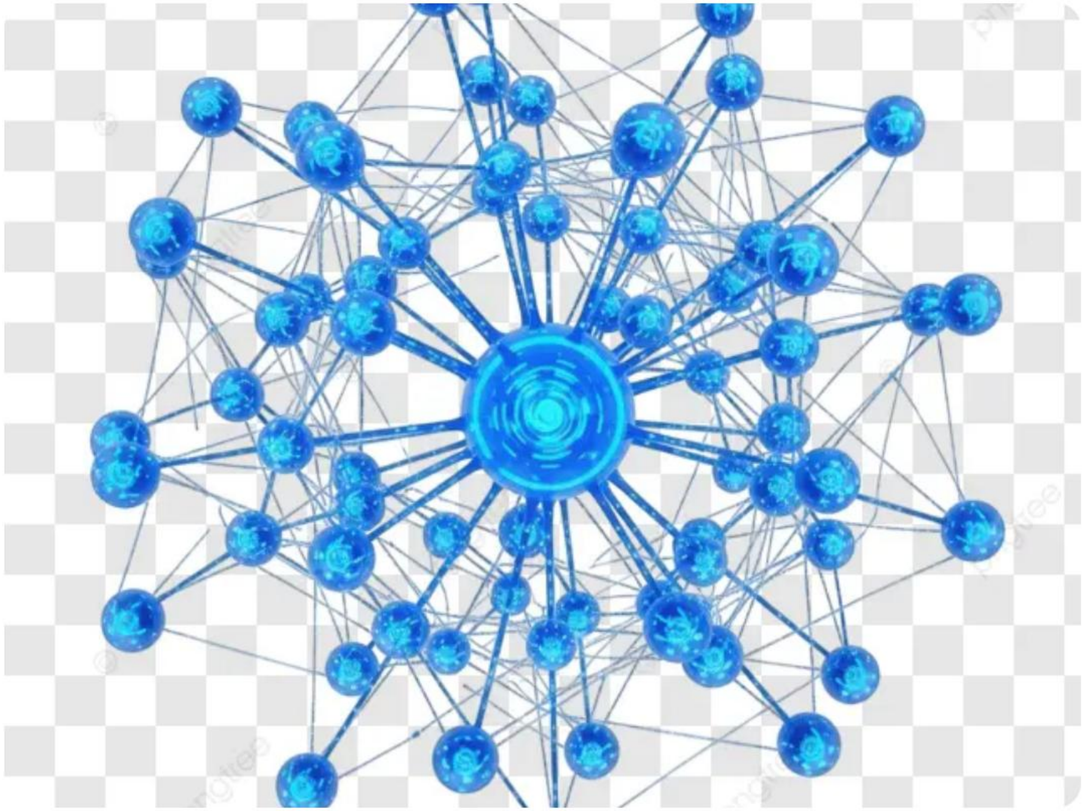
Unordered collections of properties. Perfect for modeling real-world entities.

```
{ name: "Joe", role: "Mentor" }
```

Arrays

Ordered lists of values. Flexible and dynamic in size.

```
[1, 2, 3, "JS"]
```



Core String Methods

Method	Description	Output Example
`toUpperCase()`	Converts the string to all capital letters.	"HELLO"
`slice(start, end)`	Extracts a section and returns a new string.	"ell" (from "hello", 1, 4)
`includes(substr)`	Checks if string contains the substring.	true / false
`trim()`	Removes whitespace from both ends.	"code" (from " code ")

Note: Strings in JavaScript are immutable; these methods return new strings.

MODULE 02

Control Flow

Managing the execution path of your code through logic and repetition.

Conditionals: If / Else

Standard Logic

Execute code based on a boolean condition. `if...else if...else`

Comparison

Use `===` for strict equality to avoid type coercion errors.

Ternary

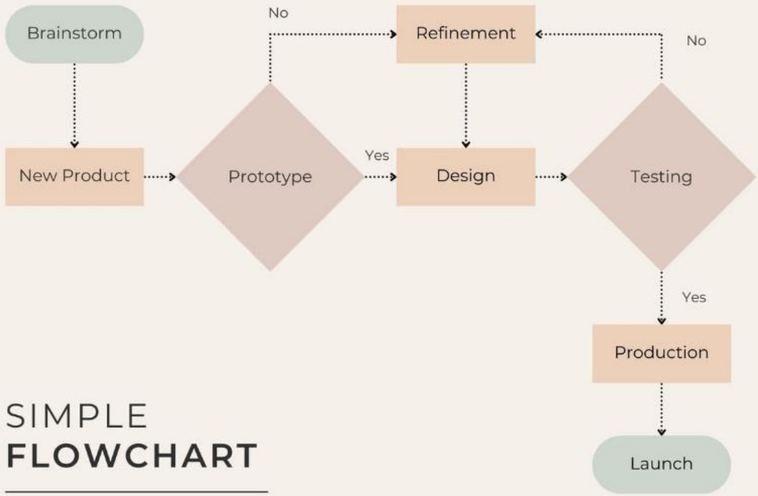
A concise one-liner for simple conditions: `cond ? true : false`

```
if (age >= 18) { console.log("Can vote"); } else { console.log("Access denied"); }
```

The Switch Statement

An elegant alternative to long `if/else` chains when checking a single value.

- **Break:** Prevents "fall-through" logic.
- **Default:** Handles cases with no matches.
- **Strict:** Uses `===` for comparisons.



Looping & Iteration

for

Counted Iteration

For Loops: Best when you know exactly how many times to repeat.

```
for (let i = 0; i < 5; i++) { ... }
```

while

Conditional Iteration

While Loops: Best when waiting for a specific state change.

```
while (loading === true) { ... }
```

MODULE 03

Functions

The building blocks of reusable, modular JavaScript code.

Anatomy of a Function

Parameters

Named variables passed into the function as placeholders.

Body

The actual logic and tasks the function performs.

Return

The output value sent back to where the function was called.

DRY Principle: Don't Repeat Yourself!

Arrow Functions

Introduced in ES6, arrow functions provide a shorter syntax and lexical `this` binding.

```
const add = (a, b) => a + b;
```

Perfect for callbacks and anonymous functions in modern development pipelines.

Best Practices

- **Clean Code:** Give functions descriptive names (e.g., `calculateTotal`).
- **Modularity:** Each function should do exactly one thing.
- **Comments:** Document complex logic for your future self.
- **Consistency:** Stick to one function style (Arrow vs Named) per project.

Questions?

Let's Debug & Discuss

joseph.kwanusu@zindua.io

zindua.io/student-portal

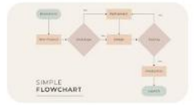
Image Sources

https://media2.dev.to/dynamic/image/width=1000,height=420,fit=cover,gravity=auto,format=auto/https%3A%2F%2Fdev-to-uploads.s3.amazonaws.com%2Fuploads%2Farticles%2Fd10yuqxe8zbpl060v2g3.jpg

Source: dev.to

https://png.pngtree.com/png-vector/20260318/ourlarge/pngtree-network-connection-visualization-png-image_18942236.webp

Source: pngtree.com



https://template.canva.com/EAGGBznhnPM/2/0/1600w-y3gbDR3e6jU.jpg

Source: www.canva.com

https://images.photowall.com/products/93727/abstract-geometric-shapes-white-on-grey.jpg?h=699&q=85

Source: www.photowall.com

https://www.cio.com/wp-content/uploads/2025/03/3485346-0-65796900-1742380165-shutterstock_2431976515.jpg?quality=50&strip=all&w=1024

Source: www.cio.com